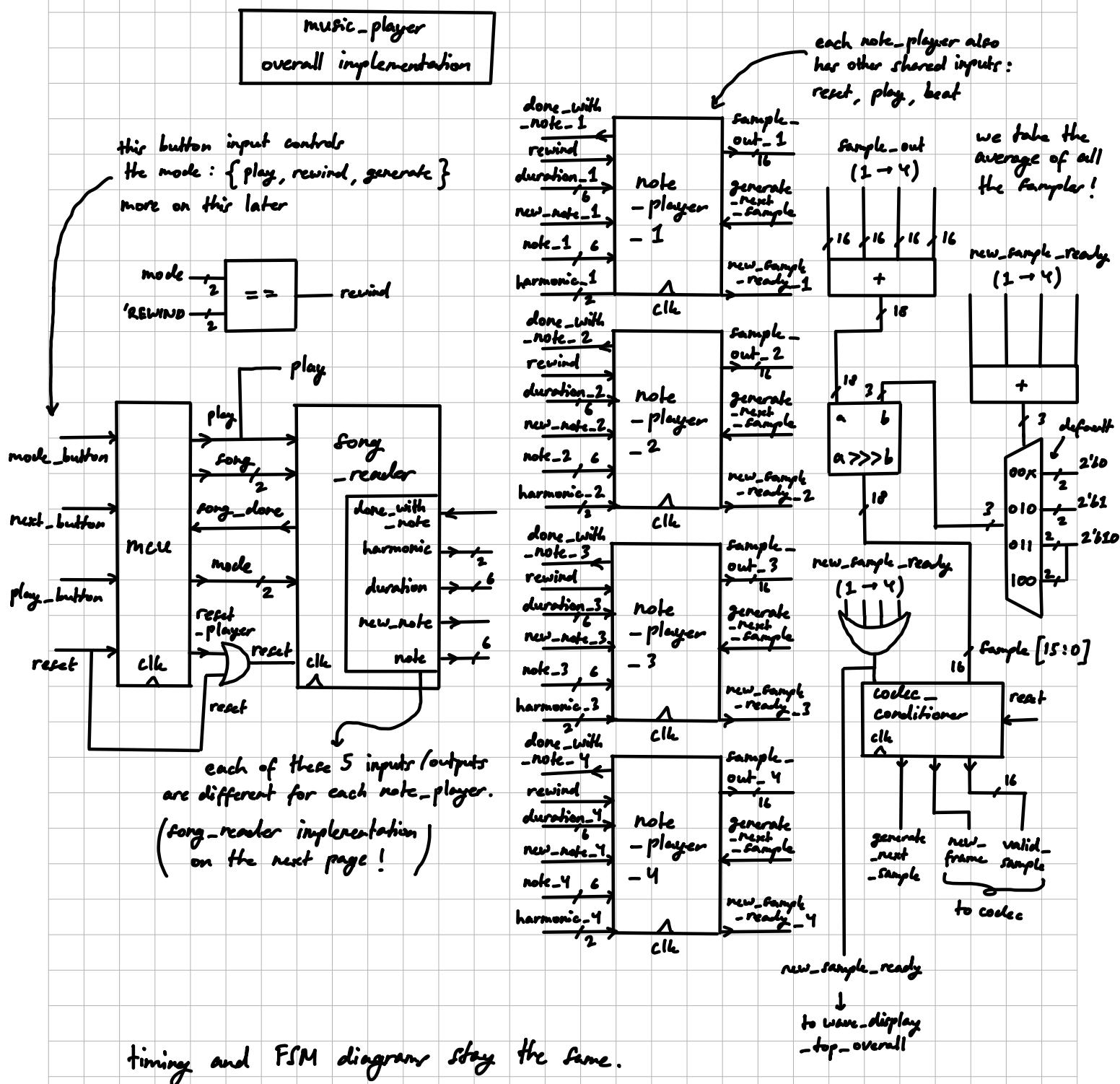


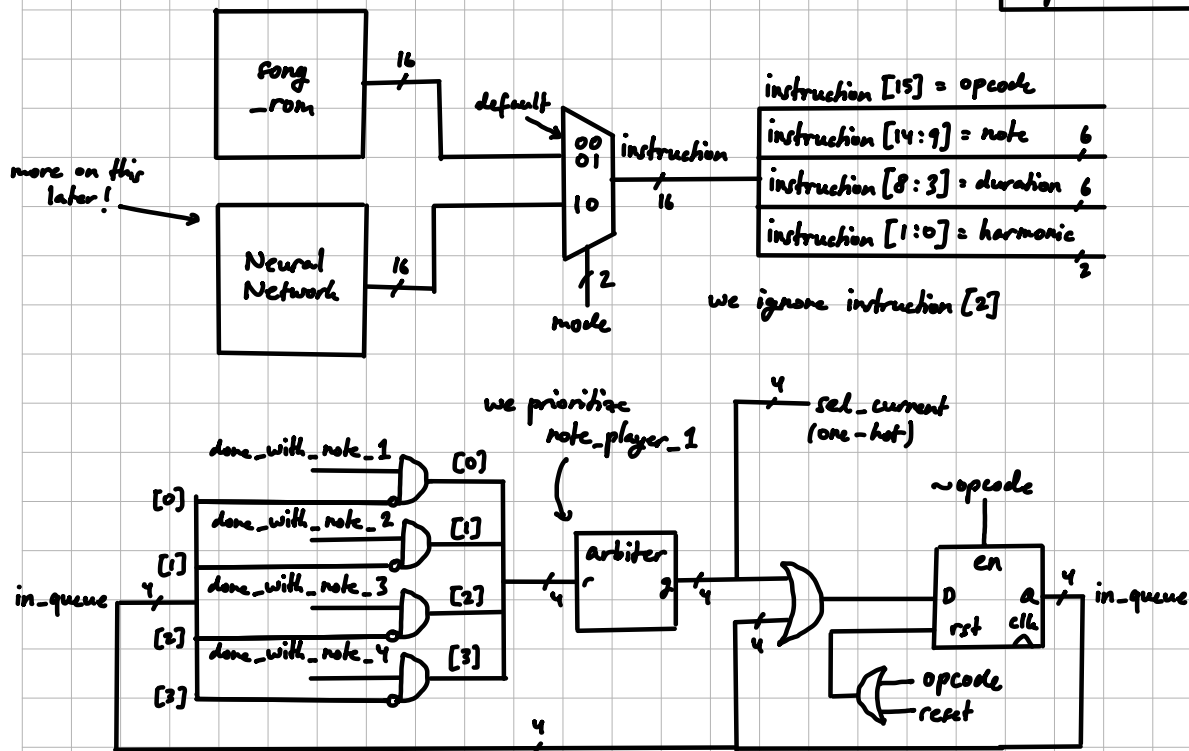
Checkpoint I

- Chords : We aim for 4 notes to be played simultaneously.
Each note played requires an additional note-player.

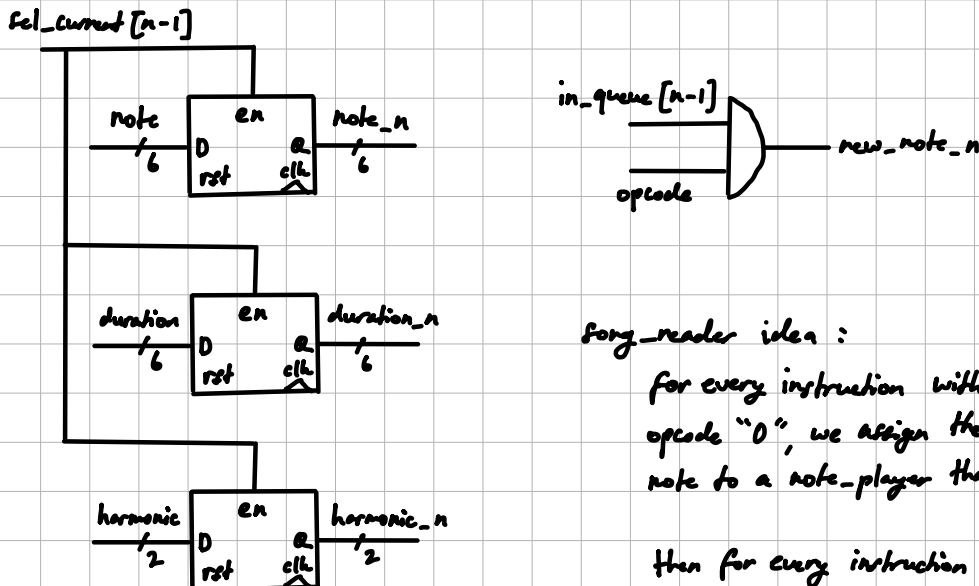


song-reader assigns signals to each note-player based on a modified song_rom which contains information about what type of note it is, its duration, and its type of harmonic.

song-reader implementation



for every note-player (replace "n" with corresponding note-player number),



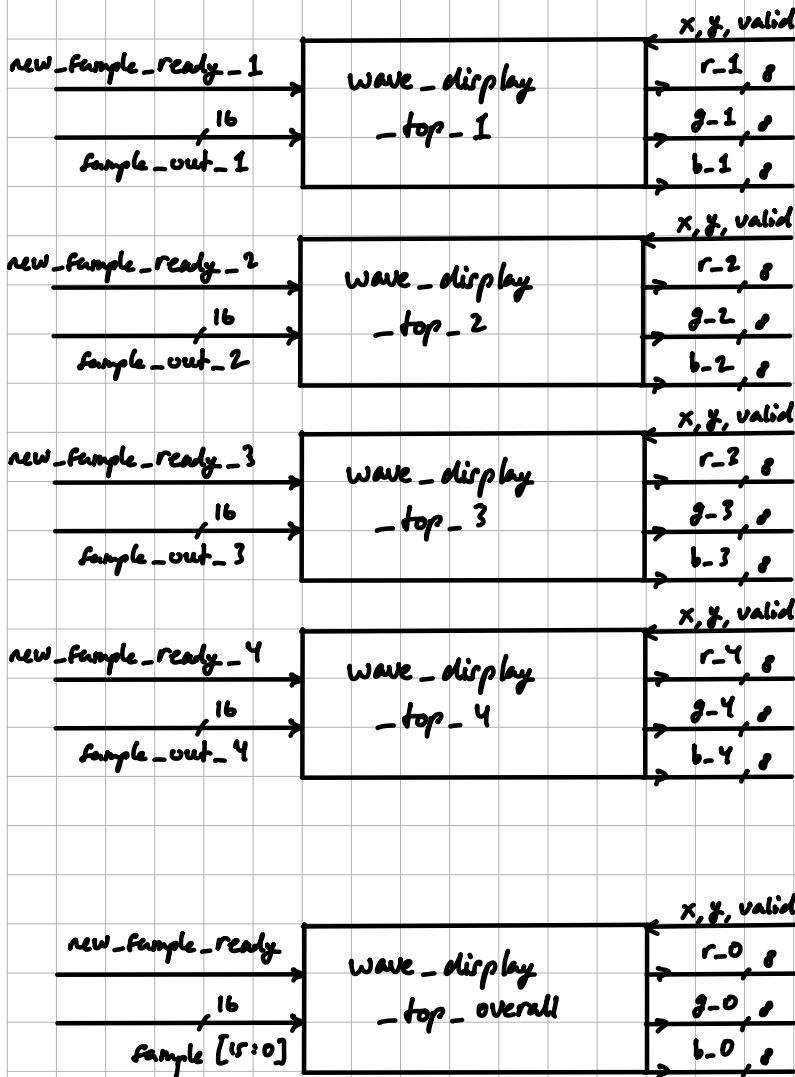
song-reader idea:

for every instruction with the opcode "0", we assign the incoming note to a note-player that is free.

then for every instruction with the opcode "1", we "flush" the new note by pulsing the signal high.

2. Enhanced Waveform Display: each of the 4 notes will be in a different color, with the combined waveform in white.

for each of the 5 total waveforms, we need to have a wave_display_top (lab5) module.

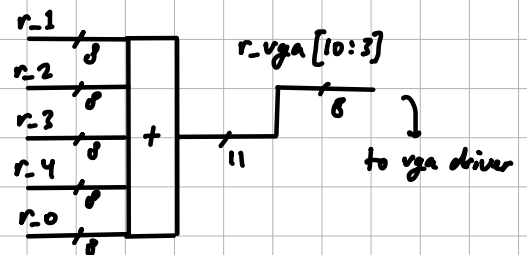


we assign `r_n, g_n, b_n` to a set value that is distinct from one another.

1 → blue
2 → red
3 → green
4 → pink
overall → white

for every single color signal (`r, g, b`), we add all 5 of them to be displayed

↳ works if there are overlapping waveforms



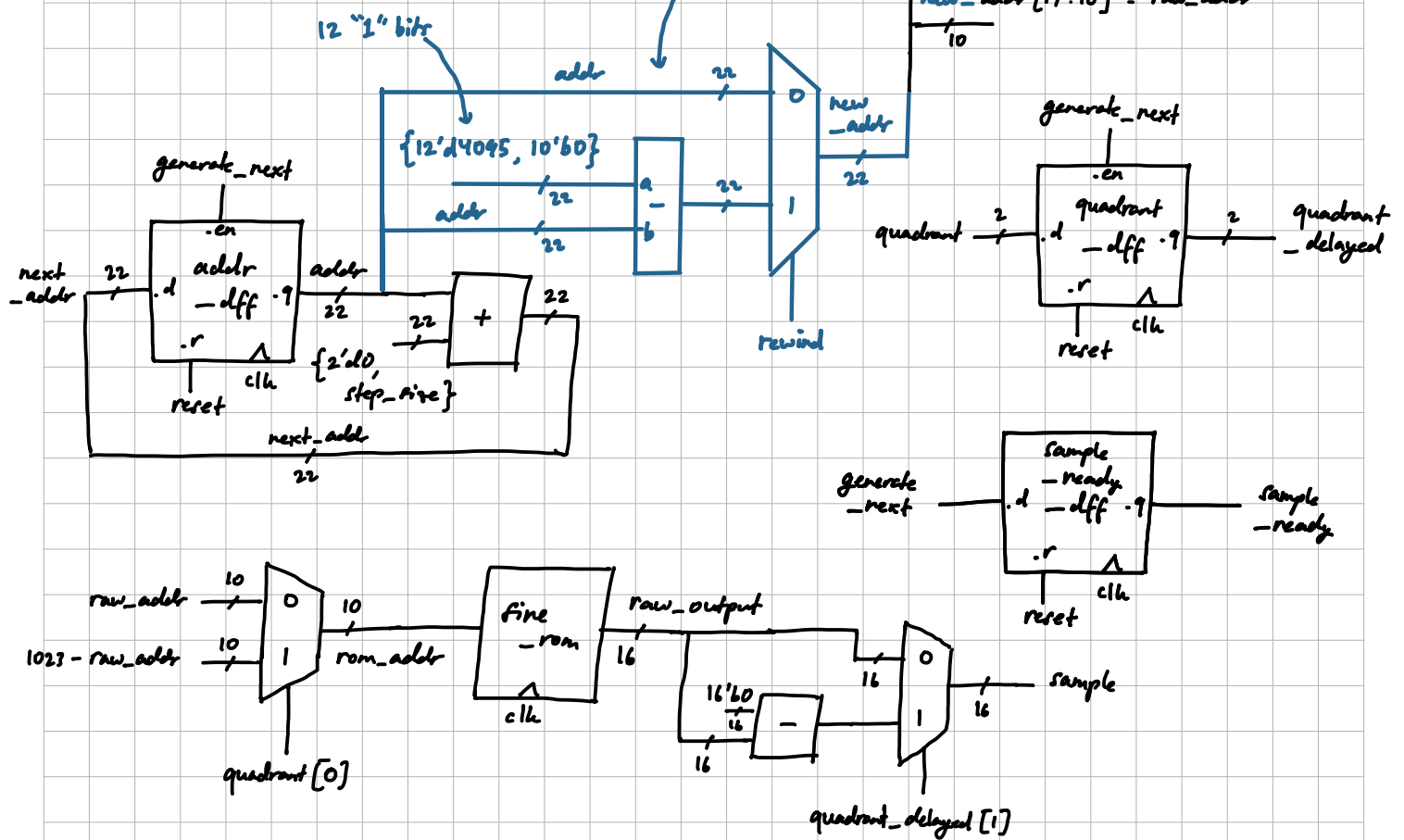
3. Rewind : We can "sample backwards" in note-player and read the notes backwards in song-reader.

line_reader → from lab4_report

we reverse the address if rewind is high

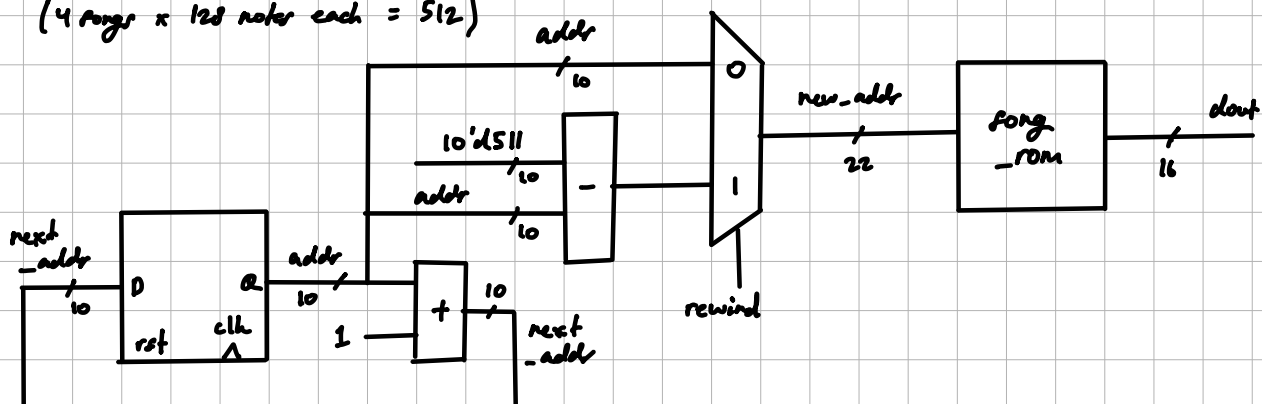
$\text{new_addr}[21:20] = \text{quadrant}$

$\text{new_addr}[19:10] = \text{raw_addr}$

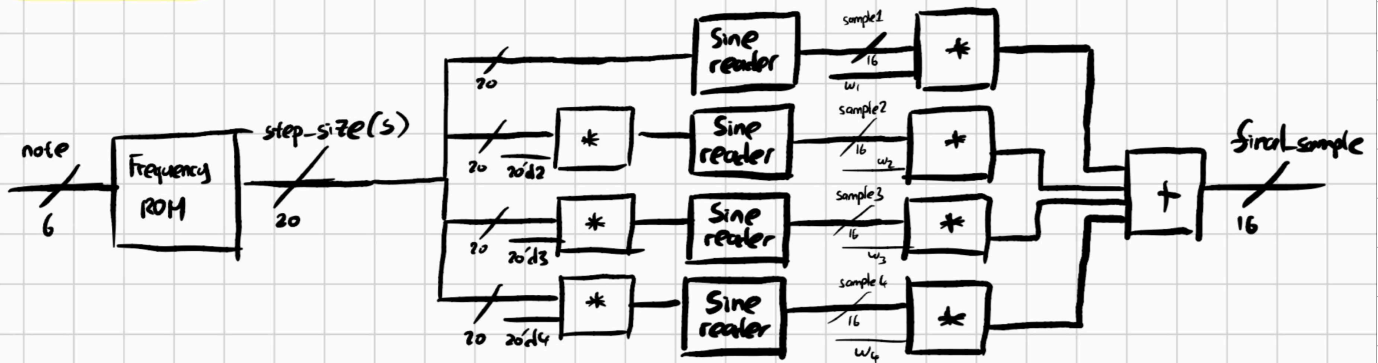


song_reader $\rightarrow$ addressing the song_ram

$(4 \text{ songs} \times 128 \text{ notes each} = 512)$



Harmonics → every note-player will have 4 sine_readers



For the weights w_1, w_2, w_3 , and w_4 , we will use:

$$w_k = \frac{1}{k}$$

To store these, we will use fixed-point:

$$\text{scale} = 2^{14} = 16384$$

$$w_k = \frac{1}{k} \times 16384$$

Therefore:

$$w_1 = 16'd16384$$

$$w_2 = 16'd8192$$

$$w_3 = 16'd5461$$

$$w_4 = 16'd4096$$

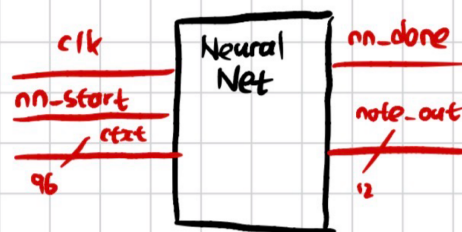
So later:

$$\text{weighted_sample}_k = (\text{sample}_k * w_k) \ggg 14$$

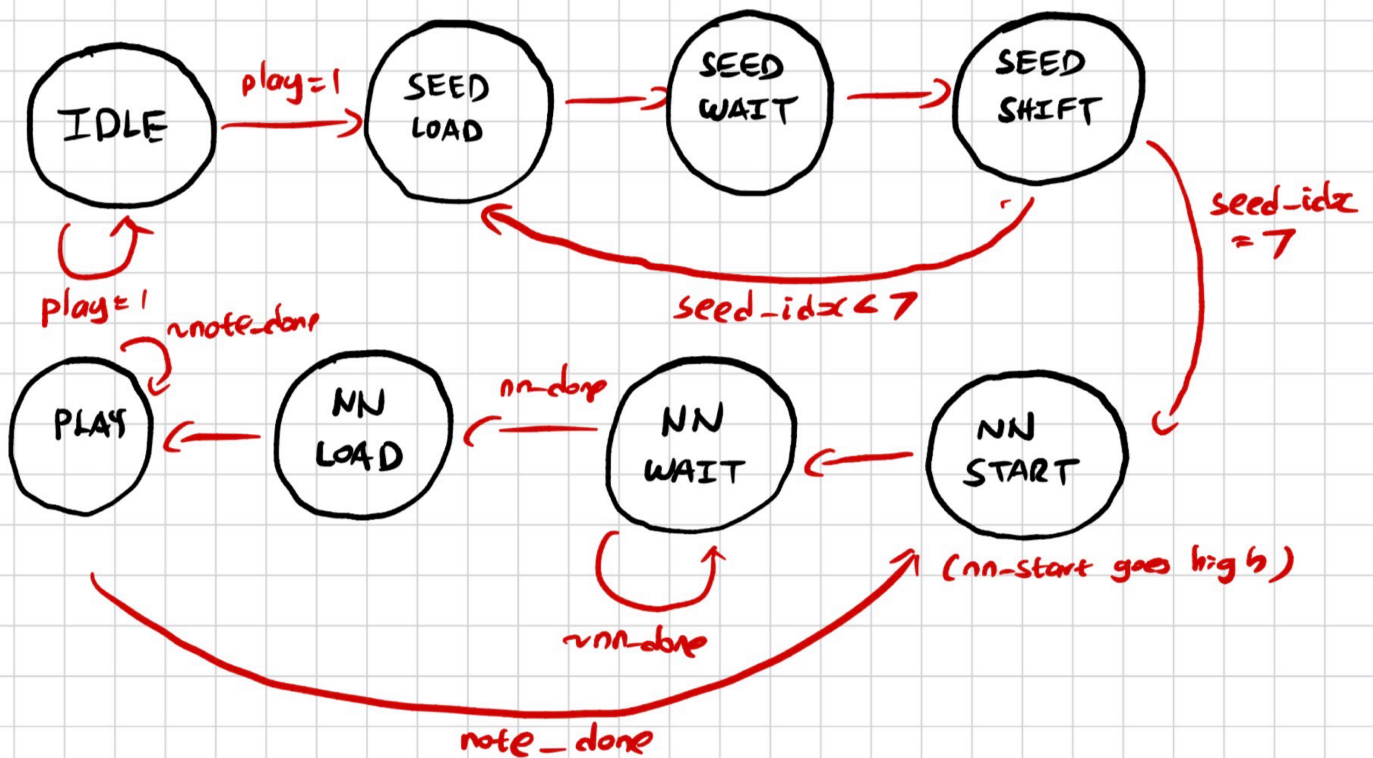
Neural Network Music Generator

Song Reader FSM

Idea: we can modify the FSM for song reader. Instead of having a WAIT state that waits for 1 cycle, it waits for a signal **nn_done** from the neural network module. before proceeding to the next state. Here is the interface for neural network module:



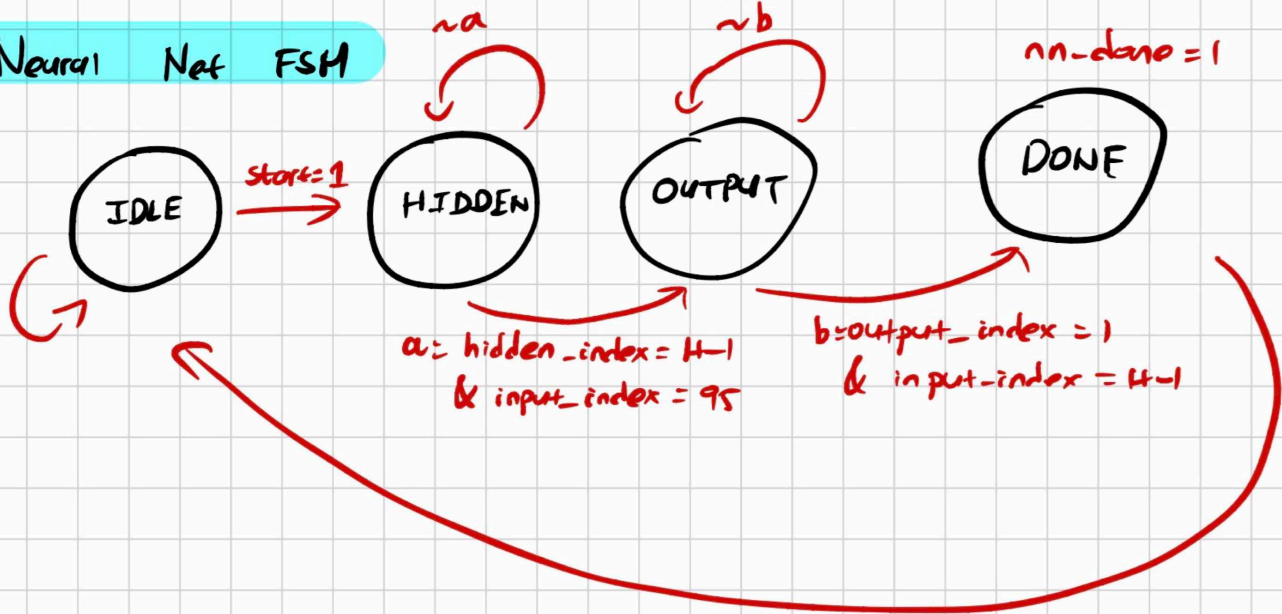
ctx is a 96-bit signal representing the last 8 notes, with the LSBs representing the most outdated note. We also create a ROM to store the first 8 notes and durations ($8 \times 12 = 96$ bits) as context for the first note generation



Neural Net module (1 hidden layer + 1 output layer)

We will have Weight ROM with size $H \times 96$ (H is the hidden layer size) and the bias ROM (both are hardcoded)

Neural Net FSM



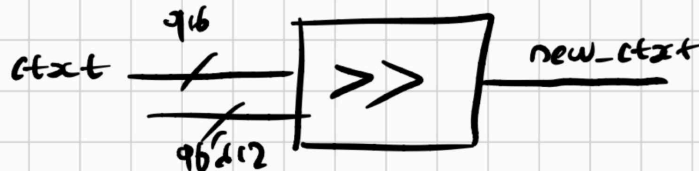
In hidden, given $x = \text{ctxt}[0 \dots 95]$, we do

$$\text{hidden_out}[h] = \text{ReLU} \left(\text{bias}[h] + \sum_{i=0}^{95} \text{weight_hidden}[h][i] \cdot \text{ctxt}[i] \right)$$

In output, we do :

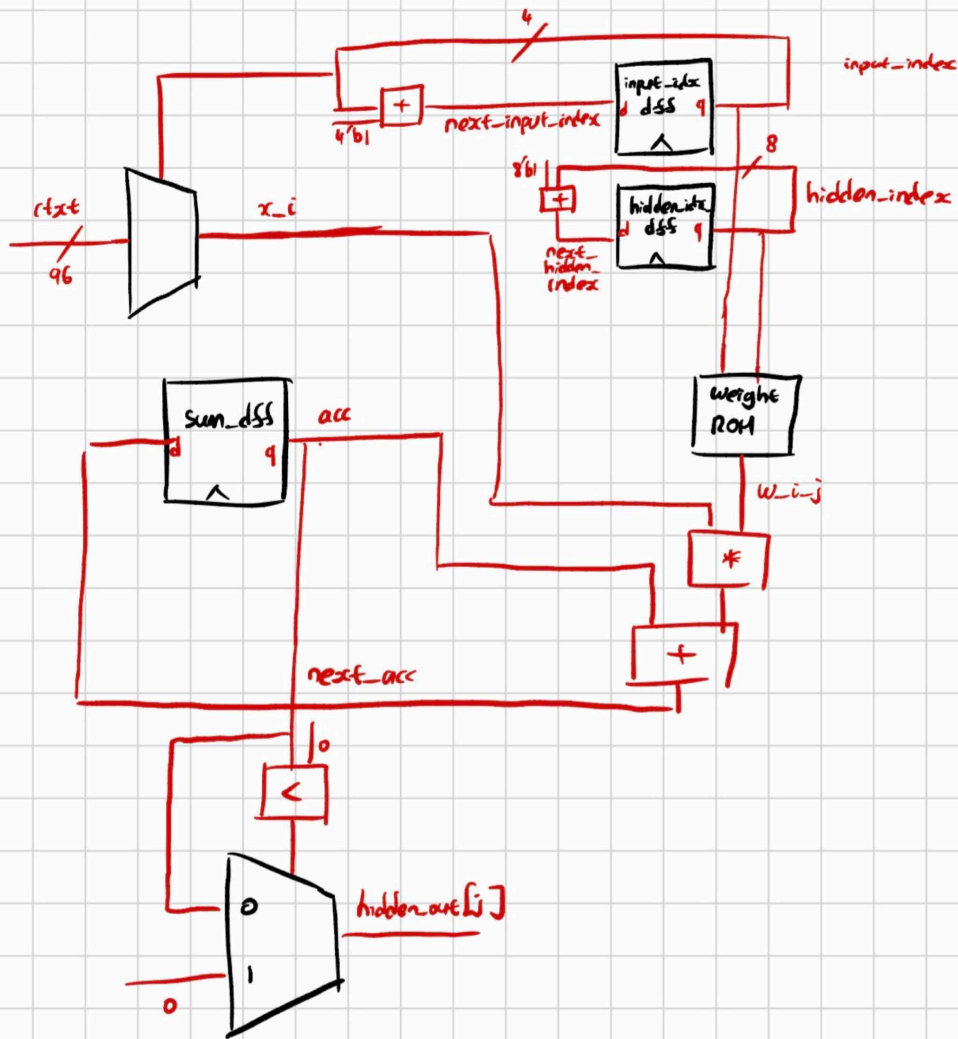
$$y[o] = \text{bias_out}[o] + \sum_{i=1}^{H-1} \text{weight_out}[o][i] \cdot \text{hidden_out}[i]$$

Given a new generated note new_note , this is how we update the context



$$\text{new_ctxt}[95:88] = \text{new_note}$$

Hidden Block



Output Block

